Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Anonymous Authors

Index Terms—

I. Preliminaries and Problem Statement

We represent a target assembly as an object-support graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $v \in \mathcal{V}$ denotes one target object instance and each directed edge $(u, v) \in \mathcal{E}$ means that object u must support object v in the final assembly. The graph is directed because support precedence is asymmetric: if $(u, v) \in \mathcal{E}$, then object u must be realized before object v can be stably placed. Optional edge attributes, such as left/right placement labels at the support interface, are kept as local geometric hints but do not by themselves define the batch sequence.

The planning goal is to convert $\mathcal{G}$ into an ordered sequence of solver-facing batches $\mathcal{B} = (B_1, \ldots, B_T)$ such that every object appears exactly once, every batch remains within the current solver scale, and every support predecessor of a node in B_t has already appeared in an earlier batch. In other words, the system must produce a decomposition that is both structurally valid and suitable for downstream geometric solving.

Directly exposing native LGP to the full graph introduces two practical bottlenecks. First, long-horizon assemblies create excessive symbolic branching, because the solver must consider many candidate symbolic configurations before geometric infeasibility can prune them. Second, mixed support patterns such as bridge-like substructures complicate the symbolic-geometric search because local support changes can affect distant parts of the same full-horizon problem. These issues motivate a decomposition stage between graph extraction and native LGP instantiation.

II. Method

Long-horizon robotic assembly requires coordinated structural and geometric reasoning, In practice, directly solving the full-horizon task in native LGP leads to high symbolic branching and poor scalability. To address this issue, we use an object-support graph as the structural interface between VLM parsing and solver instantiation. Our method follows five stages: scene grounding, support-graph construction, branch-based cutting, layer-based cutting, and selective native LGP solving. The core idea is to first represent the target assembly as a relational graph, and then decompose it into a small set of solver-facing subtasks rather than exposing the entire horizon at once.

A. Scene Grounding and Feasibility Screening

Before structural planning, the system converts the initial scene into a grounded object inventory that records object category, geometric signature, and robot-centered identity. In the same grounding stage, it also constructs a deterministic scene-order dictionary by sorting objects of the same category according to their Euclidean distance to the robot base. This dictionary is computed once from the initial scene and later reused during solver-side instantiation to bind generic object requests to concrete scene instances; it is therefore independent of the target support graph and does not define the assembly strategy itself.

To remove infeasible candidates before long-horizon planning, the system queries the waypoint-level LGP subproblem [?] as a low-cost feasibility test. For each candidate object o_i, a sparse pick-waypoint problem is solved under collision constraints within a fixed time budget T_{max}:

$$\text{Reach}(o_i) = \mathbb{I}[\exists\, q^* \text{ s.t. } \text{KOMO}_{\text{wp}}(o_i, q^*) \text{ is feasible}] \quad (1)$$

Objects that admit a feasible waypoint are retained in the grounded inventory, whereas failed or timed-out candidates are excluded from subsequent planning. In this way, the pipeline enters long-horizon planning with a solver-consistent feasibility prior obtained directly from the waypoint-level LGP subproblem, while avoiding the cost of full symbolic-geometric optimization at this stage.

B. Support Graph Construction

Instead of asking the VLM to directly output precise object coordinates or a long executable code sequence, we use it to parse the target specification into the object-support graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. This design is motivated by the capability boundary of current VLMs: they are relatively reliable at reasoning about relative spatial structure, support relations, and coarse left-right or above-below organization, but they are much less stable when required to produce exact numerical layouts, long structured programs, or mathematically precise symbolic outputs. We therefore convert the target specification into a relational graph that keeps the part VLMs are better at providing.

In this graph, each node denotes one target object instance, and each directed edge denotes one direct support relation that must hold in the assembled structure. The support graph is the central interface of the pipeline. Upstream, it converts the VLM's relational understanding

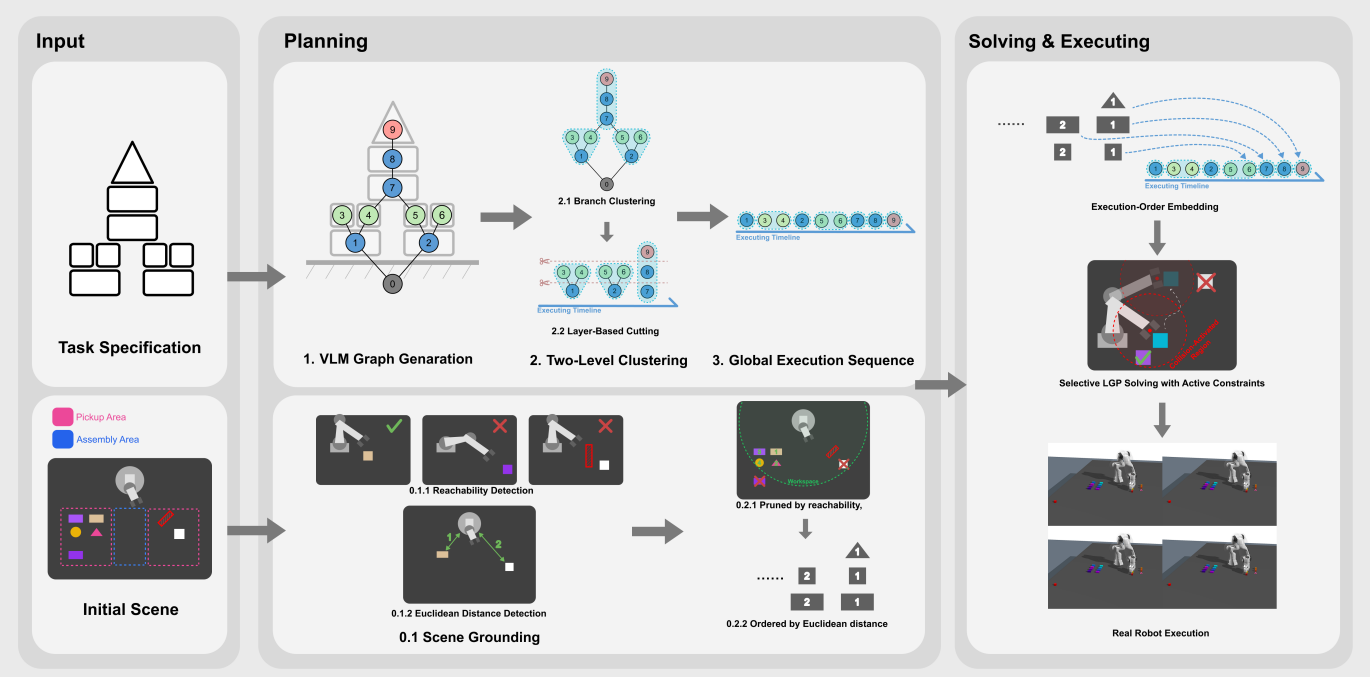

Fig. 1. The overall workflow of the proposed VLM-LGP framework.

into an explicit structural representation. Downstream, it provides the input to the decomposition module that prepares solver-consistent subproblems for native LGP. Figure ?? shows representative support graphs parsed from target specifications.

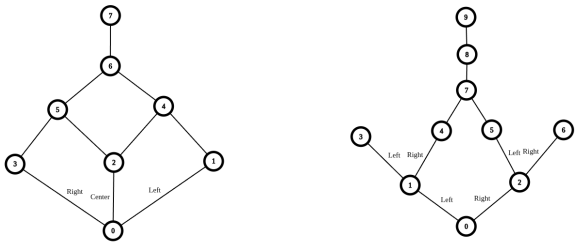

Fig. 2. Examples of object-support graphs parsed by the VLM from target assembly specifications. Directed edges encode relative support relations that are later used for branch-based and layer-based task decomposition.

C. Branch-Based Cutting

Once the task graph has been obtained, directly exposing the full graph to native LGP is undesirable from the perspective of solve speed, overall efficiency, and practical hardware limits. We therefore introduce a two-level clustering strategy to decompose the full task into smaller executable subtasks. The first level is branch-based cutting, whose goal is to identify several large structural branches before detailed execution batches are generated.

At this stage, nodes that belong to the same main support chain are grouped into the same branch, while nodes that connect multiple branches mark the transition to an upper bridge-like structure. The purpose of this step is to preserve the major structural organization of the assembly and recover a human-like construction logic by progressing from point-like supports to larger supporting surfaces while distinguishing primary versus secondary supports, instead of immediately flattening the whole graph into a single topological sequence. On the representative support graph used in this draft, the branch-based step identifies three coarse groups: the left branch 134, the right branch 256, and the upper chain 789. This result is still an intermediate structural decomposition rather than the final execution order.

D. Layer-Based Cutting

After branch-based cutting, each branch is further decomposed by a second mechanism, namely layer-based cutting. This stage is responsible for converting each large branch into a sequence of solver-facing subtasks. Inside every branch, nodes are scanned from lower support layers to higher support layers, and each local group is cut into executable batches while preserving predecessor constraints.

This second level of cutting is necessary because a branch is still too large to be sent to the solver as a single monolithic task. In the current system, and under the combined consideration of LGP solve speed and practical hardware limits, each executable batch is constrained to contain at most two placement subtasks. As a result, the decomposition not only preserves the branch structure of the target graph but also matches the scale that the downstream solver can handle efficiently.

In the running example, layer-based cutting transforms the coarse branches into the local sequences $134 \mapsto 1 \mid 34$, $256 \mapsto 2 \mid 56$, and $789 \mapsto 7 \mid 8 \mid 9$. A precedence-consistent ordering over these local results then produces the final global execution sequence $1 \mid 34 \mid 2 \mid 56 \mid 7 \mid 8 \mid 9$. This two-level decomposition is easier to explain than the previous single-stage rule set because the system first identifies large structural branches and then refines them into small executable batches.

E. Selective Native LGP Instantiation and Solving

Native LGP couples symbolic task structure with continuous trajectory optimization through a shared representation of predicates, action semantics, and decision rules [?]. In principle, this abstraction can solve assembly tasks from symbolic goals alone. In long-horizon settings, however, directly exposing the full generic task domain still leads to excessive symbolic branching and unnecessary geometric backtracking.

Our solver therefore instantiates the native LGP abstraction selectively for each execution batch. Once a batch has been determined by the two-level decomposition process, the system reads the scene-order dictionary generated during scene grounding, binds generic object categories to concrete scene instances, and compiles only the symbolic relations, object bindings, and decision rules required by the current support pattern. The solver thus receives a grounded symbolic-geometric subproblem tailored to the current batch rather than a monolithic domain covering all possible assembly alternatives.

The same waypoint-level solve also provides geometric cues for downstream motion optimization. Let x denote the full-motion trajectory optimized in the downstream stage, let $d_{ij}(x)$ denote the minimum pairwise distance between collision proxy frames f_i and f_j along x, and let f_i^w denote the pose of frame f_i at waypoint w. The downstream trajectory is obtained by solving

$$x^* = \arg\min_x J(x), \tag{2}$$

subject to collision constraints instantiated only for the active set

$$c_{ij}(x) = d_{\text{safe}} - d_{ij}(x) \le 0, \quad \forall (f_i, f_j) \in \mathcal{C}_{\text{active}}(w), \tag{3}$$

where

$$\mathcal{C}_{\text{active}}(w) = \left\{ (f_i, f_j) \;\middle|\; d(f_i^w, f_j^w) < r_{\text{active}} \right\}. \tag{4}$$

Here, $J(x)$ denotes the downstream motion objective, d_{safe} is the prescribed safety margin, and r_{active} is the activation radius used to screen candidate collision pairs from waypoint geometry. Only pairs selected by $\mathcal{C}_{\text{active}}(w)$ are instantiated as explicit collision constraints in the downstream full-motion solve. This active-constraint strategy reduces unnecessary collision checks and shortens full-motion solve time without altering the structural plan itself.

This restricted instantiation preserves the expressiveness of native LGP while concentrating the search on the structurally relevant portion of the task. It also provides a clean interface between graph-level decomposition and geometric solving: the decomposition module decides which support relations should be realized next, and native LGP resolves whether and how those relations can be executed under the current scene geometry.

F. Robot Execution

After each subproblem is solved, the resulting trajectory is parsed and dispatched to the robot controller for execution. The execution layer remains intentionally thin: it reads the planned motion and gripper commands, executes them on the real platform, and updates the scene state for the next planning-solving cycle.

References

[1] M. Toussaint, "Logic-geometric programming for multi-step manipulation," in Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI), 2015.

[2] M. Toussaint, "Newton methods for k-order Markov optimization," arXiv preprint arXiv:1407.0414, 2014.

[3] G. Karypis and V. Kumar, "Multilevel k-way partitioning scheme for irregular graphs," Journal of Parallel and Distributed Computing, vol. 48, no. 1, pp. 96–129, 1998.

[4] J. Schoeler, F. T. Rehder, and M. Toussaint, "R-LGP: Reactive Logic-Geometric Programming," arXiv preprint arXiv:2310.02791, 2023.

[5] J. Doe et al., "Receding Horizon Hierarchical Logic-Geometric Programming," arXiv preprint arXiv:2110.03420, 2021.

[6] F. Lin et al., "Text2Motion: From Natural Language to Logic-Geometric Programming," arXiv preprint arXiv:2303.12153, 2023.

[7] General VLA Reference, "Vision-Language-Action Models for Robotics," Placeholder, 2024.

[8] H. Kopka and P. W. Daly, A Guide to LaTeX, 3rd ed., 1999.

[9] M. Toussaint et al., "General VLA Reference for Task and Motion Planning," arXiv preprint arXiv:2410.02193, 2024.

[10] J. Doe et al., "Vision-Language Models for Robotic Assembly," arXiv preprint arXiv:2407.09829, 2024.

[11] Y. Zhu et al., "Fabrica: Dual-Arm Assembly of General Multi-Part Objects via Integrated Planning and Learning," arXiv preprint arXiv:2506.05168, 2024.

[12] D. McDermott et al., "PDDL-the planning domain definition language," Technical Report, Yale Center for Computational Vision and Control, 1998.